

NiceGUI 结合 JWT 实现登录认证授权

NiceGUI 是一款简洁易用的 Python Web 框架，结合 JWT (JSON Web Token) 可实现无状态的登录认证与授权。以下是完整的实现案例，包含用户登录生成 JWT、请求拦截验证 JWT、权限控制等核心功能，并附带详细代码解释。

第一步：环境准备

安装依赖包：

```
pip install nicegui pyjwt python-dotenv cryptography
```

- `pyjwt`：用于生成和解析 JWT
- `cryptography`：提供 JWT 加密算法（如 HS256）
- `python-dotenv`：管理环境变量（存储 JWT 密钥）

第二步：完整代码案例

创建 `app.py` 文件，代码结构分为：配置项、JWT 工具函数、登录页面、受保护页面、请求拦截器。

```
from nicegui import app, ui, request, Response
import jwt
import time
from dotenv import load_dotenv
import os

# 加载环境变量（建议将密钥存放在.env文件，不要硬编码）
load_dotenv()
JWT_SECRET = os.getenv("JWT_SECRET", "your-default-secret-key-123456") # 生产环境务必替换为强密钥
JWT_ALGORITHM = "HS256"
JWT_EXPIRE_SECONDS = 3600 # Token 有效期 1 小时

# 模拟用户数据库（生产环境替换为真实数据库）
USER_DB = {
    "admin": {"password": "admin123", "role": "admin"}, # 管理员权限
    "user": {"password": "user123", "role": "user"} # 普通用户权限
}

# ----- JWT 核心工具函数 -----
def generate_jwt(username: str) -> str:
    """生成 JWT Token"""
    # Payload 包含：用户名、过期时间、用户角色
    payload = {
        "sub": username,
        "exp": time.time() + JWT_EXPIRE_SECONDS,
        "role": USER_DB[username]["role"]
    }
```

```

# 生成 Token
token = jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)
return token

def verify_jwt(token: str) -> dict | None:
    """验证 JWT Token, 返回解析后的 payload (验证失败返回 None)"""
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
        # 额外验证: 用户名是否存在于用户数据库
        if payload["sub"] not in USER_DB:
            return None
        return payload
    except jwt.ExpiredSignatureError:
        # Token 过期
        return None
    except jwt.InvalidTokenError:
        # Token 无效 (签名错误、格式错误等)
        return None

# ----- 页面路由与认证拦截 -----
@app.middleware("http")
async def auth_middleware(request: request, call_next):
    """
    全局认证中间件:
    - 放行登录页 (/login) 和静态资源
    - 其他页面需验证 JWT Token (从 Cookie 或 Header 获取)
    """
    # 无需认证的路径
    if request.url.path in ["/", "/login"] or request.url.path.startswith("/_nicegui"):
        response = await call_next(request)
        return response

    # 从 Cookie 获取 Token (也可从 Header: Authorization 中获取)
    token = request.cookies.get("auth_token")
    if not token:
        # 无 Token, 重定向到登录页
        response = Response(status_code=307)
        response.headers["Location"] = "/login"
        return response

    # 验证 Token
    payload = verify_jwt(token)
    if not payload:
        # Token 无效/过期, 清除 Cookie 并重定向到登录页
        response = Response(status_code=307)
        response.headers["Location"] = "/login"
        response.delete_cookie("auth_token")
        return response

    # 将用户信息存入请求上下文, 供后续页面使用
    app.storage.user.update({"username": payload["sub"], "role": payload["role"]})
    response = await call_next(request)
    return response

```

```

# ----- 登录页面 -----
@ui.page("/login")
def login_page():
    """登录页面：验证用户名密码，生成 JWT 并写入 Cookie"""
    ui.label("用户登录").style("font-size: 24px; font-weight: bold; margin-bottom: 20px;")

    # 输入框
    username_input = ui.input("用户名").style("width: 300px; margin-bottom: 10px;")
    password_input = ui.input("密码").password().style("width: 300px; margin-bottom: 20px;")
    # 错误提示
    error_label = ui.label("").style("color: red; margin-bottom: 10px;")

    def handle_login():
        """登录按钮点击事件"""
        username = username_input.value.strip()
        password = password_input.value.strip()

        # 验证用户名密码
        if username not in USER_DB or USER_DB[username]["password"] != password:
            error_label.set_text("用户名或密码错误!")
            return

        # 生成 JWT Token
        token = generate_jwt(username)
        # 将 Token 写入 Cookie (设置 HttpOnly 增强安全性，生产环境建议加 Secure)
        ui.run_javascript(f'document.cookie = "auth_token={token}"; path=/; max-age={JWT_EXPIRE_SECONDS};')
        # 重定向到首页
        ui.navigate.to("/home")

    # 登录按钮
    ui.button("登录", on_click=handle_login).style("width: 300px;")

# ----- 受保护的首页 (普通用户/管理员均可访问) -----
@ui.page("/home")
def home_page():
    """首页：展示用户信息，提供登出功能"""
    username = app.storage.user.get("username")
    role = app.storage.user.get("role")

    ui.label(f"欢迎回来, {username} ({role})!").style("font-size: 20px; margin-bottom: 20px;")

    def handle_logout():
        """登出：清除 Cookie 和用户信息，重定向到登录页"""
        ui.run_javascript('document.cookie = "auth_token=; path=/; max-age=0;')
        app.storage.user.clear()
        ui.navigate.to("/login")

    ui.button("登出", on_click=handle_logout).style("margin-top: 10px;")

    # 权限控制：仅管理员可见的按钮

```

```
if role == "admin":
    ui.button("管理员专属操作", on_click=lambda: ui.notify("管理员操作执行成功!")).style("margin-top: 10px;")

# ----- 启动应用 -----
if __name__ in {"__main__", "__mp_main__"}:
    ui.run(title="NiceGUI JWT 认证示例", port=8080)
```

第三步：代码关键解释

1. JWT 核心函数

- `generate_jwt`：接收用户名，生成包含**用户名 (sub)**、**过期时间 (exp)**、**角色 (role)** 的 JWT Token，使用 HS256 算法加密。
- `verify_jwt`：解析并验证 Token，捕获「过期」「无效」两类异常，验证通过后返回解析后的 payload。

2. 全局认证中间件

- 作用：拦截所有 HTTP 请求，对非登录页的请求验证 Token。
- 逻辑：
 - 放行登录页 (`/login`) 和 NiceGUI 内置静态资源 (`/_nicegui`)；
 - 从 Cookie 中获取 `auth_token` (也可从 `Authorization` 请求头获取，如 `Bearer <token>`)；
 - Token 无效 / 过期时，清除 Cookie 并重定向到登录页；
 - 验证通过后，将用户信息存入 `app.storage.user` (NiceGUI 内置的用户上下文)。

3. 登录页面

- 验证用户名密码是否匹配模拟数据库；
- 验证通过后生成 Token，通过 JavaScript 将 Token 写入 Cookie (设置 `max-age` 与 Token 有效期一致)；
- 重定向到受保护的首页。

4. 权限控制

- 在首页中，通过 `app.storage.user.get("role")` 判断用户角色，仅管理员显示「管理员专属操作」按钮。

第四步：运行与测试

1. 创建 `.env` 文件 (可选，用于存储密钥)：

```
JWT_SECRET=my-strong-secret-key-888888 # 替换为随机强密钥
```

1. 运行 `app.py`：

```
python app.py
```

1. 访问 `http://localhost:8080/login`：

- 管理员账号: `admin` / `admin123` → 可看到「管理员专属操作」按钮;
- 普通用户账号: `user` / `user123` → 无管理员按钮;
- 错误账号: 提示「用户名或密码错误」;
- 直接访问 `http://localhost:8080/home`: 重定向到登录页。

第五步：生产环境优化建议

1. 密钥安全: 不要硬编码 JWT 密钥, 使用环境变量或配置中心管理;
2. 密码存储: 不要明文存储密码, 使用 `bcrypt` 等算法加密后存储;
3. Cookie 安全: 生产环境设置 `Secure=True` (仅 HTTPS 传输)、`HttpOnly=True` (防止 XSS 攻击)、`SameSite=Strict`;
4. Token 传递: 除 Cookie 外, 也可通过 `Authorization: Bearer <token>` 请求头传递 (更适合前后端分离场景);
5. 刷新 Token: 实现「刷新 Token」机制, 避免用户频繁登录;
6. 数据库: 替换模拟用户数据库为 MySQL/PostgreSQL 等真实数据库。

补充：基于 JWT 的刷新 Token 机制实现

刷新 Token (Refresh Token) 的核心逻辑是:

1. 生成「访问 Token (Access Token)」(短期有效, 如 1 小时) 和「刷新 Token (Refresh Token)」(长期有效, 如 7 天);
2. 访问 Token 过期后, 用户无需重新登录, 通过刷新 Token 获取新的访问 Token;
3. 刷新 Token 仅用于获取新的访问 Token, 不直接用于接口授权。

以下是完整的迭代代码, 在原有基础上新增刷新 Token 功能:

第一步：更新依赖（无需新增，保持原有依赖即可）

```
pip install nicegui pyjwt python-dotenv cryptography
```

第二步：完整迭代代码

```
from nicegui import app, ui, request, Response
import jwt
import time
from dotenv import load_dotenv
import os
import uuid # 用于生成唯一的刷新Token ID

# 加载环境变量
load_dotenv()
JWT_SECRET = os.getenv("JWT_SECRET", "your-default-secret-key-123456")
```

```

JWT_ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE = 3600 # 访问Token有效期: 1小时
REFRESH_TOKEN_EXPIRE = 60 * 60 * 24 * 7 # 刷新Token有效期: 7天

# 模拟用户数据库
USER_DB = {
    "admin": {"password": "admin123", "role": "admin"},
    "user": {"password": "user123", "role": "user"}
}

# 模拟刷新Token存储（生产环境替换为Redis/数据库，设置过期时间）
REFRESH_TOKEN_STORE = {} # 结构: {refresh_token_id: (username, expire_time)}

# ----- JWT 工具函数（新增刷新Token逻辑） -----
def generate_tokens(username: str) -> tuple[str, str]:
    """生成访问Token和刷新Token"""
    # 1. 生成访问Token（Payload包含用户名、角色、过期时间）
    access_payload = {
        "sub": username,
        "type": "access",
        "role": USER_DB[username]["role"],
        "exp": time.time() + ACCESS_TOKEN_EXPIRE
    }
    access_token = jwt.encode(access_payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

    # 2. 生成刷新Token（Payload包含唯一ID、用户名、过期时间）
    refresh_token_id = str(uuid.uuid4()) # 唯一ID, 用于标识刷新Token
    refresh_payload = {
        "jti": refresh_token_id,
        "sub": username,
        "type": "refresh",
        "exp": time.time() + REFRESH_TOKEN_EXPIRE
    }
    refresh_token = jwt.encode(refresh_payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

    # 3. 存储刷新Token（生产环境用Redis，设置过期时间）
    REFRESH_TOKEN_STORE[refresh_token_id] = (username, refresh_payload["exp"])

    return access_token, refresh_token

def verify_token(token: str, token_type: str = "access") -> dict | None:
    """验证Token（区分访问/刷新Token）"""
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
        # 验证Token类型和用户名
        if payload.get("type") != token_type or payload["sub"] not in USER_DB:
            return None
        # 刷新Token额外验证: 存储中是否存在且未过期
        if token_type == "refresh":
            jti = payload.get("jti")
            if jti not in REFRESH_TOKEN_STORE:
                return None
            _, expire_time = REFRESH_TOKEN_STORE[jti]

```

```

        if time.time() > expire_time:
            del REFRESH_TOKEN_STORE[jti] # 清理过期刷新Token
            return None
        return payload
    except jwt.ExpiredSignatureError:
        return None
    except jwt.InvalidTokenError:
        return None

def refresh_access_token(refresh_token: str) -> str | None:
    """通过刷新Token获取新的访问Token"""
    # 验证刷新Token
    refresh_payload = verify_token(refresh_token, token_type="refresh")
    if not refresh_payload:
        return None
    username = refresh_payload["sub"]
    # 生成新的访问Token（刷新Token仍沿用，直到过期）
    access_payload = {
        "sub": username,
        "type": "access",
        "role": USER_DB[username]["role"],
        "exp": time.time() + ACCESS_TOKEN_EXPIRE
    }
    new_access_token = jwt.encode(access_payload, JWT_SECRET, algorithm=JWT_ALGORITHM)
    return new_access_token

# ----- 全局中间件（适配刷新Token） -----
@app.middleware("http")
async def auth_middleware(request: request, call_next):
    # 放行登录页、刷新Token接口、静态资源
    if request.url.path in ["/", "/login", "/refresh-token"] or \
    request.url.path.startswith("/_nicegui"):
        response = await call_next(request)
        return response

    # 从Cookie获取访问Token
    access_token = request.cookies.get("access_token")
    if not access_token:
        # 无访问Token，重定向登录
        response = Response(status_code=307)
        response.headers["Location"] = "/login"
        return response

    # 验证访问Token
    payload = verify_token(access_token)
    if payload:
        # 访问Token有效，存入用户上下文
        app.storage.user.update({"username": payload["sub"], "role": payload["role"]})
        response = await call_next(request)
        return response
    else:
        # 访问Token过期/无效，尝试用刷新Token刷新
        refresh_token = request.cookies.get("refresh_token")

```

```

    if not refresh_token:
        # 无刷新Token, 重定向登录
        response = Response(status_code=307)
        response.headers["Location"] = "/login"
        response.delete_cookie("access_token")
        response.delete_cookie("refresh_token")
        return response

    # 刷新访问Token
    new_access_token = refresh_access_token(refresh_token)
    if not new_access_token:
        # 刷新Token无效, 重定向登录
        response = Response(status_code=307)
        response.headers["Location"] = "/login"
        response.delete_cookie("access_token")
        response.delete_cookie("refresh_token")
        return response

    # 刷新成功, 写入新的访问Token到Cookie
    response = await call_next(request)
    response.set_cookie(
        key="access_token",
        value=new_access_token,
        path="/",
        max_age=ACCESS_TOKEN_EXPIRE,
        httponly=True # 生产环境加 secure=True (HTTPS)
    )
    # 解析新Token, 存入用户上下文
    new_payload = verify_token(new_access_token)
    app.storage.user.update({"username": new_payload["sub"], "role":
new_payload["role"]})
    return response

# ----- 刷新Token接口 -----
@ui.page("/refresh-token")
async def refresh_token_api():
    """刷新Token接口 (供前端主动调用)"""
    refresh_token = request.cookies.get("refresh_token")
    if not refresh_token:
        return Response(status_code=401, content="无刷新Token")

    new_access_token = refresh_access_token(refresh_token)
    if not new_access_token:
        return Response(status_code=401, content="刷新Token无效/过期")

    # 返回新的访问Token (也可直接写入Cookie)
    response = Response(status_code=200, content={"access_token": new_access_token})
    response.set_cookie(
        key="access_token",
        value=new_access_token,
        path="/",
        max_age=ACCESS_TOKEN_EXPIRE,
        httponly=True

```



```

    )
    return response

# ----- 登录页面（更新为双Token） -----
@ui.page("/login")
def login_page():
    ui.label("用户登录").style("font-size: 24px; font-weight: bold; margin-bottom: 20px;")
    username_input = ui.input("用户名").style("width: 300px; margin-bottom: 10px;")
    password_input = ui.input("密码").password().style("width: 300px; margin-bottom: 20px;")
    error_label = ui.label("").style("color: red; margin-bottom: 10px;")

    def handle_login():
        username = username_input.value.strip()
        password = password_input.value.strip()

        if username not in USER_DB or USER_DB[username]["password"] != password:
            error_label.set_text("用户名或密码错误！")
            return

        # 生成双Token
        access_token, refresh_token = generate_tokens(username)
        # 写入Cookie
        ui.run_javascript(f"""
            document.cookie = "access_token={access_token}; path=/; max-age=
{ACCESS_TOKEN_EXPIRE}; httponly;";
            document.cookie = "refresh_token={refresh_token}; path=/; max-age=
{REFRESH_TOKEN_EXPIRE}; httponly;";
            """)
        # 重定向首页
        ui.navigate.to("/home")

    ui.button("登录", on_click=handle_login).style("width: 300px;")

# ----- 首页（新增登出逻辑：清理刷新Token存储） -----
@ui.page("/home")
def home_page():
    username = app.storage.user.get("username")
    role = app.storage.user.get("role")

    ui.label(f"欢迎回来, {username} ({role}) !").style("font-size: 20px; margin-bottom: 20px;")

    def handle_logout():
        """登出：清理Cookie和刷新Token存储"""
        # 1. 清除Cookie
        ui.run_javascript("""
            document.cookie = "access_token=; path=/; max-age=0;";
            document.cookie = "refresh_token=; path=/; max-age=0;";
            """)
        # 2. 解析刷新Token并从存储中删除（可选，增强安全性）
        refresh_token = request.cookies.get("refresh_token")
        if refresh_token:
            try:

```

```

        refresh_payload = jwt.decode(refresh_token, JWT_SECRET, algorithms=
[JWT_ALGORITHM])
        jti = refresh_payload.get("jti")
        if jti in REFRESH_TOKEN_STORE:
            del REFRESH_TOKEN_STORE[jti]
        except:
            pass
        # 3. 清空用户上下文并重定向
        app.storage.user.clear()
        ui.navigate.to("/login")

    ui.button("登出", on_click=handle_logout).style("margin-top: 10px;")

    if role == "admin":
        ui.button("管理员专属操作", on_click=lambda: ui.notify("管理员操作执行成
功! ")).style("margin-top: 10px;")

# ----- 启动应用 -----
if __name__ in {"__main__", "__mp_main__"}:
    ui.run(title="NiceGUI JWT 刷新Token示例", port=8080)

```

核心逻辑解释

1. 双 Token 生成

- `generate_tokens` 函数同时生成「访问 Token」（短期）和「刷新 Token」（长期）；
- 刷新 Token 包含唯一 ID（`jti`），并存储在 `REFRESH_TOKEN_STORE` 中（生产环境建议用 Redis，支持过期自动清理）。

2. Token 验证优化

- `verify_token` 函数新增 `token_type` 参数，区分验证「访问 Token」和「刷新 Token」；
- 刷新 Token 验证时，额外检查存储中的有效性（防止伪造 / 重复使用）。

3. 自动刷新机制

- 中间件检测到访问 Token 过期后，自动读取刷新 Token，调用 `refresh_access_token` 生成新的访问 Token；
- 新访问 Token 直接写入 Cookie，用户无感知，无需重新登录。

4. 主动刷新接口

- 新增 `/refresh-token` 接口，供前端主动调用（如页面切换时提前刷新 Token）；
- 接口返回新的访问 Token，也可直接写入 Cookie。

5. 登出逻辑增强

- 登出时不仅清除 Cookie，还删除存储中的刷新 Token，防止刷新 Token 被滥用。

测试流程

- 登录后，手动修改 Cookie 中的 `access_token` 为过期值（或等待 1 小时）；
- 刷新首页，中间件会自动用刷新 Token 生成新的访问 Token，页面正常显示；
- 若刷新 Token 也过期（7 天后），则重定向到登录页；
- 登出后，刷新 Token 会从存储中删除，无法再用于刷新。

生产环境补充建议

- 刷新 Token 存储：用 Redis 替代内存字典，设置与刷新 Token 一致的过期时间，自动清理无效 Token；
- 刷新 Token 防复用：刷新 Token 使用一次后立即失效（生成新的刷新 Token），防止刷新 Token 被盗用；
- 接口限流：对 `/refresh-token` 接口做限流，防止暴力请求；
- HTTPS：生产环境必须启用 HTTPS，Cookie 设置 `secure=True`，防止 Token 被劫持。

迭代：基于 Redis 存储刷新 Token + 防复用实现

Redis 是实现刷新 Token 存储的最佳选择（支持过期自动清理、高性能、分布式），「防复用」核心是：**刷新 Token 使用一次后立即失效，并生成新的刷新 Token**，避免刷新 Token 被盗用后多次刷新访问 Token。

第一步：环境准备

新增 Redis 依赖：

```
pip install redis
```

确保本地 / 服务器已启动 Redis 服务（默认端口 6379）。

第二步：完整代码（集成 Redis + 防复用）

```
from nicegui import app, ui, request, Response
import jwt
import time
from dotenv import load_dotenv
import os
import uuid
import redis  # 新增Redis依赖

# 加载环境变量
load_dotenv()
JWT_SECRET = os.getenv("JWT_SECRET", "your-default-secret-key-123456")
```

```

JWT_ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE = 3600 # 访问Token: 1小时
REFRESH_TOKEN_EXPIRE = 60 * 60 * 24 * 7 # 刷新Token: 7天

# ----- Redis 初始化 -----
# 连接Redis (生产环境建议配置密码、集群、超时等)
REDIS_CLIENT = redis.Redis(
    host=os.getenv("REDIS_HOST", "localhost"),
    port=int(os.getenv("REDIS_PORT", 6379)),
    db=int(os.getenv("REDIS_DB", 0)),
    # password=os.getenv("REDIS_PASSWORD", ""), # 生产环境添加密码
    decode_responses=True # 自动将返回值转为字符串
)

# 模拟用户数据库
USER_DB = {
    "admin": {"password": "admin123", "role": "admin"},
    "user": {"password": "user123", "role": "user"}
}

# ----- JWT 工具函数 (适配Redis+防复用) -----
def generate_tokens(username: str) -> tuple[str, str]:
    """生成访问Token+刷新Token (刷新Token存入Redis, 防复用)"""
    # 1. 生成访问Token
    access_payload = {
        "sub": username,
        "type": "access",
        "role": USER_DB[username]["role"],
        "exp": time.time() + ACCESS_TOKEN_EXPIRE
    }
    access_token = jwt.encode(access_payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

    # 2. 生成刷新Token (唯一ID+用户名+过期时间)
    refresh_token_id = str(uuid.uuid4()) # 唯一标识
    refresh_payload = {
        "jti": refresh_token_id,
        "sub": username,
        "type": "refresh",
        "exp": time.time() + REFRESH_TOKEN_EXPIRE
    }
    refresh_token = jwt.encode(refresh_payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

    # 3. 刷新Token存入Redis (key: refresh:{jti}, value: username, 过期时间与刷新Token一致)
    redis_key = f"refresh:{refresh_token_id}"
    REDIS_CLIENT.setex(
        name=redis_key,
        time=REFRESH_TOKEN_EXPIRE,
        value=username
    )

    return access_token, refresh_token

def verify_token(token: str, token_type: str = "access") -> dict | None:

```

```

"""验证Token（区分类型，刷新Token需校验Redis）"""
try:
    payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
    # 基础验证：类型、用户名
    if payload.get("type") != token_type or payload["sub"] not in USER_DB:
        return None

    # 刷新Token额外验证：Redis中存在且未过期
    if token_type == "refresh":
        jti = payload.get("jti")
        redis_key = f"refresh:{jti}"
        # 检查Redis中是否存在该刷新Token
        username_in_redis = REDIS_CLIENT.get(redis_key)
        if not username_in_redis or username_in_redis != payload["sub"]:
            return None

    return payload
except jwt.ExpiredSignatureError:
    return None
except jwt.InvalidTokenError:
    return None

def refresh_access_token(refresh_token: str) -> tuple[str | None, str | None]:
    """
    刷新访问Token（防复用逻辑）：
    - 验证旧刷新Token，有效则删除（防止复用）
    - 生成新的访问Token+刷新Token
    - 返回：（新访问Token，新刷新Token）
    """
    # 1. 验证旧刷新Token
    refresh_payload = verify_token(refresh_token, token_type="refresh")
    if not refresh_payload:
        return None, None

    # 2. 防复用：删除旧刷新Token（使用一次即失效）
    old_jti = refresh_payload["jti"]
    old_redis_key = f"refresh:{old_jti}"
    REDIS_CLIENT.delete(old_redis_key)

    # 3. 生成新的访问Token+刷新Token
    username = refresh_payload["sub"]
    new_access_token, new_refresh_token = generate_tokens(username)

    return new_access_token, new_refresh_token

# ----- 全局中间件（适配Redis+新刷新Token） -----
@app.middleware("http")
async def auth_middleware(request: request, call_next):
    # 放行白名单
    if request.url.path in ["/", "/login", "/refresh-token"] or
    request.url.path.startswith("/_nicegui"):
        response = await call_next(request)
        return response

```

```

# 获取访问Token
access_token = request.cookies.get("access_token")
if not access_token:
    response = Response(status_code=307, headers={"Location": "/login"})
    return response

# 验证访问Token
payload = verify_token(access_token)
if payload:
    app.storage.user.update({"username": payload["sub"], "role": payload["role"]})
    response = await call_next(request)
    return response
else:
    # 访问Token失效, 尝试刷新
    refresh_token = request.cookies.get("refresh_token")
    if not refresh_token:
        # 无刷新Token, 重定向登录+清理Cookie
        response = Response(status_code=307, headers={"Location": "/login"})
        response.delete_cookie("access_token")
        response.delete_cookie("refresh_token")
        return response

    # 刷新Token (返回新的访问Token+刷新Token)
    new_access_token, new_refresh_token = refresh_access_token(refresh_token)
    if not new_access_token or not new_refresh_token:
        # 刷新Token无效, 重定向登录
        response = Response(status_code=307, headers={"Location": "/login"})
        response.delete_cookie("access_token")
        response.delete_cookie("refresh_token")
        return response

    # 刷新成功: 写入新Token到Cookie
    response = await call_next(request)
    # 新访问Token
    response.set_cookie(
        key="access_token",
        value=new_access_token,
        path="/",
        max_age=ACCESS_TOKEN_EXPIRE,
        httponly=True,
        # secure=True # 生产环境HTTPS开启
    )
    # 新刷新Token (防复用后生成的新Token)
    response.set_cookie(
        key="refresh_token",
        value=new_refresh_token,
        path="/",
        max_age=REFRESH_TOKEN_EXPIRE,
        httponly=True,
        # secure=True # 生产环境HTTPS开启
    )
    # 更新用户上下文

```

```

        new_payload = verify_token(new_access_token)
        app.storage.user.update({"username": new_payload["sub"], "role":
new_payload["role"]})
        return response

# ----- 刷新Token接口（适配新逻辑） -----
@ui.page("/refresh-token")
async def refresh_token_api():
    """主动刷新Token接口"""
    refresh_token = request.cookies.get("refresh_token")
    if not refresh_token:
        return Response(status_code=401, content="无刷新Token")

    new_access_token, new_refresh_token = refresh_access_token(refresh_token)
    if not new_access_token or not new_refresh_token:
        return Response(status_code=401, content="刷新Token无效/已使用过")

    # 返回新Token并写入Cookie
    response = Response(
        status_code=200,
        content={"access_token": new_access_token, "refresh_token": new_refresh_token}
    )
    response.set_cookie("access_token", new_access_token, path="/",
max_age=ACCESS_TOKEN_EXPIRE, httponly=True)
    response.set_cookie("refresh_token", new_refresh_token, path="/",
max_age=REFRESH_TOKEN_EXPIRE, httponly=True)
    return response

# ----- 登录/首页/登出（适配Redis） -----
@ui.page("/login")
def login_page():
    ui.label("用户登录（Redis+防复用）").style("font-size: 24px; font-weight: bold; margin-
bottom: 20px;")
    username_input = ui.input("用户名").style("width: 300px; margin-bottom: 10px;")
    password_input = ui.input("密码").password().style("width: 300px; margin-bottom: 20px;")
    error_label = ui.label("").style("color: red; margin-bottom: 10px;")

    def handle_login():
        username = username_input.value.strip()
        password = password_input.value.strip()
        if username not in USER_DB or USER_DB[username]["password"] != password:
            error_label.set_text("用户名或密码错误！")
            return

        # 生成双Token
        access_token, refresh_token = generate_tokens(username)
        # 写入Cookie
        ui.run_javascript(f"""
            document.cookie = "access_token={access_token}; path=/; max-age=
{ACCESS_TOKEN_EXPIRE}; httponly;";
            document.cookie = "refresh_token={refresh_token}; path=/; max-age=
{REFRESH_TOKEN_EXPIRE}; httponly;";
            """)

```

```

        ui.navigate.to("/home")

    ui.button("登录", on_click=handle_login).style("width: 300px;")

@ui.page("/home")
def home_page():
    username = app.storage.user.get("username")
    role = app.storage.user.get("role")

    ui.label(f"欢迎回来, {username} ({role}) !").style("font-size: 20px; margin-bottom: 20px;")

    def handle_logout():
        """登出: 清理Cookie+Redis中的刷新Token"""
        # 1. 清除Cookie
        ui.run_javascript("""
            document.cookie = "access_token=; path=/; max-age=0;";
            document.cookie = "refresh_token=; path=/; max-age=0;";
        """)
        # 2. 清理Redis中的刷新Token
        refresh_token = request.cookies.get("refresh_token")
        if refresh_token:
            try:
                refresh_payload = jwt.decode(refresh_token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
                jti = refresh_payload.get("jti")
                redis_key = f"refresh:{jti}"
                REDIS_CLIENT.delete(redis_key)
            except:
                pass
        # 3. 清空上下文并重定向
        app.storage.user.clear()
        ui.navigate.to("/login")

    ui.button("登出", on_click=handle_logout).style("margin-top: 10px;")
    if role == "admin":
        ui.button("管理员专属操作", on_click=lambda: ui.notify("管理员操作执行成功!")).style("margin-top: 10px;")

# ----- 启动应用 -----
if __name__ in {"__main__", "__mp_main__"}:
    ui.run(title="NiceGUI JWT Redis 防复用示例", port=8080)

```

核心逻辑解释

1. Redis 集成

- 初始化 Redis 客户端, 连接本地 Redis 服务 (生产环境需配置密码、集群、超时等) ;
- 刷新 Token 存储格式: `key=refresh:{jti}` (jti 为刷新 Token 唯一 ID) , `value=用户名` , `过期时间=刷新Token有效期` , Redis 会自动清理过期的刷新 Token。

2. 防复用核心逻辑

- `refresh_access_token` 函数中，验证旧刷新 Token 有效后，**立即删除 Redis 中的旧刷新 Token**，确保该 Token 只能使用一次；
- 删除旧 Token 后，生成新的访问 Token + 刷新 Token，返回给客户端并写入 Cookie；
- 若旧刷新 Token 已被使用过（Redis 中无记录），则刷新失败，强制用户重新登录。

3. 登出增强

- 登出时不仅清除 Cookie，还删除 Redis 中对应的刷新 Token，彻底失效所有未过期的刷新 Token。

4. 安全性提升

- 刷新 Token 使用一次即失效，即使 Token 被盗，攻击者也只能使用一次，大幅降低安全风险；
- Redis 自动过期清理，无需手动维护无效 Token。

测试防复用效果

1. 登录后，复制 Cookie 中的 `refresh_token`；
2. 手动调用 `/refresh-token` 接口（如用 Postman），使用该 `refresh_token`，首次调用会返回新 Token，Redis 中旧 Token 被删除；
3. 再次用同一个旧 `refresh_token` 调用接口，会返回「刷新 Token 无效 / 已使用过」，验证防复用生效。

生产环境优化

1. Redis 配置：
 - 启用密码认证，避免未授权访问；
 - 配置 Redis 集群，提高可用性；
 - 对刷新 Token 的 Key 设置前缀（如 `prod:refresh:{jti}`），区分环境。
2. 异常处理：
 - 增加 Redis 连接失败的降级策略（如临时使用内存存储）；
 - 对 JWT 解析、Redis 操作添加日志记录，便于排查问题。
3. 性能优化：
 - 对 `/refresh-token` 接口添加限流（如每分钟最多 5 次），防止暴力请求；
 - 缓存用户信息，减少数据库查询。

迭代：密码加密存储（bcrypt）+ 登录失败次数限制

本次迭代核心优化：

1. **密码加密**：使用 bcrypt 算法哈希存储密码（不可逆），替代明文存储；
2. **登录限流**：限制单用户登录失败次数（如 5 次），防止暴力破解；

3. **Redis 扩展**：存储登录失败次数，支持分布式环境下的限流。

第一步：环境准备

新增 bcrypt 依赖：

```
pip install bcrypt
```

确保 Redis 服务正常运行（延续上一轮的 Redis 配置）。

第二步：完整代码（集成 bcrypt + 登录限流）

```
from nicegui import app, ui, request, Response
import jwt
import time
from dotenv import load_dotenv
import os
import uuid
import redis
import bcrypt # 新增bcrypt依赖

# 加载环境变量
load_dotenv()
JWT_SECRET = os.getenv("JWT_SECRET", "your-default-secret-key-123456")
JWT_ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE = 3600 # 访问Token: 1小时
REFRESH_TOKEN_EXPIRE = 60 * 60 * 24 * 7 # 刷新Token: 7天

# ----- 配置项 -----
MAX_LOGIN_FAILED = 5 # 单用户最大登录失败次数
LOGIN_LOCK_SECONDS = 300 # 登录失败超限后锁定时长（5分钟）

# ----- Redis 初始化 -----
REDIS_CLIENT = redis.Redis(
    host=os.getenv("REDIS_HOST", "localhost"),
    port=int(os.getenv("REDIS_PORT", 6379)),
    db=int(os.getenv("REDIS_DB", 0)),
    decode_responses=True
)

# ----- 加密用户数据库（替代明文） -----
# 生成bcrypt哈希密码的工具函数（首次初始化用户时使用）
def hash_password(password: str) -> str:
    """将明文密码转为bcrypt哈希值"""
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password.encode("utf-8"), salt)
    return hashed.decode("utf-8")

# 模拟加密后的用户数据库（生产环境存储到MySQL/PostgreSQL）
# 初始密码：admin→admin123, user→user123（已哈希）
```

```

USER_DB = {
    "admin": {
        "password_hash": hash_password("admin123"), # 哈希后的值, 不可逆
        "role": "admin"
    },
    "user": {
        "password_hash": hash_password("user123"),
        "role": "user"
    }
}

# ----- 登录限流工具函数 -----
def get_login_failed_count(username: str) -> int:
    """获取用户登录失败次数"""
    redis_key = f"login:failed:{username}"
    count = REDIS_CLIENT.get(redis_key)
    return int(count) if count else 0

def incr_login_failed_count(username: str):
    """增加登录失败次数, 设置锁定时长"""
    redis_key = f"login:failed:{username}"
    # 自增失败次数, 设置过期时间 (锁定时长)
    REDIS_CLIENT.incr(redis_key)
    REDIS_CLIENT.expire(redis_key, LOGIN_LOCK_SECONDS)

def reset_login_failed_count(username: str):
    """登录成功后重置失败次数"""
    redis_key = f"login:failed:{username}"
    REDIS_CLIENT.delete(redis_key)

def is_user_locked(username: str) -> bool:
    """检查用户是否因登录失败过多被锁定"""
    count = get_login_failed_count(username)
    return count >= MAX_LOGIN_FAILED

# ----- JWT 工具函数 (沿用前版) -----
def generate_tokens(username: str) -> tuple[str, str]:
    access_payload = {
        "sub": username,
        "type": "access",
        "role": USER_DB[username]["role"],
        "exp": time.time() + ACCESS_TOKEN_EXPIRE
    }
    access_token = jwt.encode(access_payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

    refresh_token_id = str(uuid.uuid4())
    refresh_payload = {
        "jti": refresh_token_id,
        "sub": username,
        "type": "refresh",
        "exp": time.time() + REFRESH_TOKEN_EXPIRE
    }
    refresh_token = jwt.encode(refresh_payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

```

```

redis_key = f"refresh:{refresh_token_id}"
REDIS_CLIENT.setex(redis_key, REFRESH_TOKEN_EXPIRE, username)

return access_token, refresh_token

def verify_token(token: str, token_type: str = "access") -> dict | None:
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
        if payload.get("type") != token_type or payload["sub"] not in USER_DB:
            return None

        if token_type == "refresh":
            jti = payload.get("jti")
            redis_key = f"refresh:{jti}"
            username_in_redis = REDIS_CLIENT.get(redis_key)
            if not username_in_redis or username_in_redis != payload["sub"]:
                return None

        return payload
    except jwt.ExpiredSignatureError:
        return None
    except jwt.InvalidTokenError:
        return None

def refresh_access_token(refresh_token: str) -> tuple[str | None, str | None]:
    refresh_payload = verify_token(refresh_token, token_type="refresh")
    if not refresh_payload:
        return None, None

    old_jti = refresh_payload["jti"]
    old_redis_key = f"refresh:{old_jti}"
    REDIS_CLIENT.delete(old_redis_key)

    username = refresh_payload["sub"]
    new_access_token, new_refresh_token = generate_tokens(username)

    return new_access_token, new_refresh_token

# ----- 全局中间件（沿用前版） -----
@app.middleware("http")
async def auth_middleware(request: request, call_next):
    if request.url.path in ["/", "/login", "/refresh-token"] or request.url.path.startswith("/_nicegui"):
        response = await call_next(request)
        return response

    access_token = request.cookies.get("access_token")
    if not access_token:
        response = Response(status_code=307, headers={"Location": "/login"})
        return response

    payload = verify_token(access_token)

```

```

if payload:
    app.storage.user.update({"username": payload["sub"], "role": payload["role"]})
    response = await call_next(request)
    return response
else:
    refresh_token = request.cookies.get("refresh_token")
    if not refresh_token:
        response = Response(status_code=307, headers={"Location": "/login"})
        response.delete_cookie("access_token")
        response.delete_cookie("refresh_token")
        return response

    new_access_token, new_refresh_token = refresh_access_token(refresh_token)
    if not new_access_token or not new_refresh_token:
        response = Response(status_code=307, headers={"Location": "/login"})
        response.delete_cookie("access_token")
        response.delete_cookie("refresh_token")
        return response

    response = await call_next(request)
    response.set_cookie(
        key="access_token",
        value=new_access_token,
        path="/",
        max_age=ACCESS_TOKEN_EXPIRE,
        httponly=True
    )
    response.set_cookie(
        key="refresh_token",
        value=new_refresh_token,
        path="/",
        max_age=REFRESH_TOKEN_EXPIRE,
        httponly=True
    )
    new_payload = verify_token(new_access_token)
    app.storage.user.update({"username": new_payload["sub"], "role":
new_payload["role"]})
    return response

# ----- 刷新Token接口（沿用前版） -----
@ui.page("/refresh-token")
async def refresh_token_api():
    refresh_token = request.cookies.get("refresh_token")
    if not refresh_token:
        return Response(status_code=401, content="无刷新Token")

    new_access_token, new_refresh_token = refresh_access_token(refresh_token)
    if not new_access_token or not new_refresh_token:
        return Response(status_code=401, content="刷新Token无效/已使用过")

    response = Response(
        status_code=200,
        content={"access_token": new_access_token, "refresh_token": new_refresh_token}
    )

```

```

    )
    response.set_cookie("access_token", new_access_token, path="/",
max_age=ACCESS_TOKEN_EXPIRE, httponly=True)
    response.set_cookie("refresh_token", new_refresh_token, path="/",
max_age=REFRESH_TOKEN_EXPIRE, httponly=True)
    return response

# ----- 登录页面（新增加密+限流逻辑） -----
@ui.page("/login")
def login_page():
    ui.label("用户登录（bcrypt+登录限流）").style("font-size: 24px; font-weight: bold; margin-
bottom: 20px;")
    username_input = ui.input("用户名").style("width: 300px; margin-bottom: 10px;")
    password_input = ui.input("密码").password().style("width: 300px; margin-bottom: 20px;")
    error_label = ui.label("").style("color: red; margin-bottom: 10px;")
    lock_label = ui.label("").style("color: orange; margin-bottom: 10px;")

    def handle_login():
        """登录逻辑：先检查锁定→验证密码→更新失败次数"""
        username = username_input.value.strip()
        password = password_input.value.strip()

        # 清空提示
        error_label.set_text("")
        lock_label.set_text("")

        # 1. 检查用户是否存在
        if username not in USER_DB:
            error_label.set_text("用户名不存在！")
            return

        # 2. 检查用户是否被锁定
        if is_user_locked(username):
            remain_seconds = REDIS_CLIENT.ttl(f"login:failed:{username}")
            lock_label.set_text(f"登录失败次数过多，账号已锁定！剩余解锁时间：{remain_seconds}秒")
            return

        # 3. 验证密码（bcrypt比对）
        password_hash = USER_DB[username]["password_hash"].encode("utf-8")
        if not bcrypt.checkpw(password.encode("utf-8"), password_hash):
            # 密码错误，增加失败次数
            incr_login_failed_count(username)
            remain_count = MAX_LOGIN_FAILED - get_login_failed_count(username)
            error_label.set_text(f"密码错误！剩余尝试次数：{remain_count}次")
            return

        # 4. 登录成功：重置失败次数+生成Token
        reset_login_failed_count(username)
        access_token, refresh_token = generate_tokens(username)
        # 写入Cookie
        ui.run_javascript(f"""
            document.cookie = "access_token={access_token}; path=/; max-age=
{ACCESS_TOKEN_EXPIRE}; httponly;"";

```

```

        document.cookie = "refresh_token={refresh_token}; path=/; max-age=
{REFRESH_TOKEN_EXPIRE}; httponly;";
        """)
        ui.navigate.to("/home")

    ui.button("登录", on_click=handle_login).style("width: 300px;")

# ----- 首页/登出（沿用前版） -----
@ui.page("/home")
def home_page():
    username = app.storage.user.get("username")
    role = app.storage.user.get("role")

    ui.label(f"欢迎回来, {username} ({role}) !").style("font-size: 20px; margin-bottom:
20px;")

    def handle_logout():
        # 清除Cookie
        ui.run_javascript("""
            document.cookie = "access_token=; path=/; max-age=0;";
            document.cookie = "refresh_token=; path=/; max-age=0;";
            """)
        # 清理Redis中的刷新Token
        refresh_token = request.cookies.get("refresh_token")
        if refresh_token:
            try:
                refresh_payload = jwt.decode(refresh_token, JWT_SECRET, algorithms=
[JWT_ALGORITHM])
                jti = refresh_payload.get("jti")
                redis_key = f"refresh:{jti}"
                REDIS_CLIENT.delete(redis_key)
            except:
                pass
        # 清空上下文并重定向
        app.storage.user.clear()
        ui.navigate.to("/login")

    ui.button("登出", on_click=handle_logout).style("margin-top: 10px;")
    if role == "admin":
        ui.button("管理员专属操作", on_click=lambda: ui.notify("管理员操作执行成
功! ")).style("margin-top: 10px;")

# ----- 启动应用 -----
if __name__ in {"__main__", "__mp_main__"}:
    ui.run(title="NiceGUI JWT bcrypt 登录限流示例", port=8080)

```

核心逻辑解释

1. 密码加密 (bcrypt)

- `hash_password` 函数：使用 `bcrypt` 生成随机盐值，将明文密码哈希为不可逆的字符串（每次哈希结果不同，但验证时可比对）；
- 密码验证：通过 `bcrypt.checkpw` 比对明文密码和存储的哈希值，无需解密（`bcrypt` 自动处理盐值）；
- 生产环境：首次创建用户时，调用 `hash_password` 存储哈希值，永远不存储明文密码。

2. 登录失败次数限制

- **Redis 存储失败次数**：Key 格式 `login:failed:{username}`，值为失败次数，过期时间为锁定时长（5 分钟）；
- `is_user_locked`：检查失败次数是否超过阈值（5 次），超过则锁定用户；
- `incr_login_failed_count`：密码错误时自增失败次数，设置过期时间（锁定时长）；
- `reset_login_failed_count`：登录成功后删除 Redis 中的失败次数记录，重置锁定状态。

3. 登录页面逻辑优化

- 先检查用户是否存在 → 检查是否被锁定 → 验证密码 → 成功则重置失败次数，失败则增加次数并提示剩余尝试次数；
- 锁定时提示剩余解锁时间（Redis 的 TTL 命令获取 Key 剩余过期时间）。

测试流程

1. 启动 Redis 和应用，访问 `http://localhost:8080/login`；
2. 输入正确用户名 (admin/user) + 错误密码，连续 5 次：
 - 第 1-4 次：提示「密码错误！剩余尝试次数：X 次」；
 - 第 5 次：提示「登录失败次数过多，账号已锁定！剩余解锁时间：XXX 秒」；
3. 锁定期间无法登录，等待 5 分钟后解锁；
4. 输入正确密码 (admin123/user123)：登录成功，失败次数重置。

生产环境补充建议

1. 密码策略：强制用户设置复杂密码（长度≥8、包含大小写 / 数字 / 特殊字符）；
2. 锁定通知：用户被锁定时，发送邮件 / 短信提醒；
3. 日志记录：记录登录失败 / 锁定 / 解锁日志，便于审计；
4. 批量操作防护：对用户注册 / 登录接口添加 IP 限流，防止批量暴力破解；
5. `bcrypt` 优化：调整工作因子（默认 12），平衡加密强度和性能。